Royaume du Maroc
☆☆☆
Haut Commissariat au Plan
☆☆☆
Institut National de
Statistique et d'Economie
Appliquée

Gestion d'un Cabinet de Médecine Générale

Réalisé par :

Elkihel El Mehdi, El Assri El Houcein, Charaf Said, Imad mrimi

Table des matières

1 Cahier des Charges **3**

1.1 Explicitation des Besoins Clients 3

1.1.1 Besoins Fonctionnels (Spécifications) 3

1.1.2 Besoins Non-Fonctionnels 3

1.2 Choix du Modèle pour le Cycle de Vie 4

1.2.1 Présentation du modèle choisi 4

1.2.2 Pourquoi ce modèle est adapté au projet ? 4

1.2.3 Description des étapes selon le modèle incrémental 4

1.3 Planning Prévisionnel 5

2 Conception via des AGL et Documentation **6**

2.0.1 Modélisation UML 6

2.0.2 Modèle physique de données 15

3 Rapport de résultats du projet **17**

3.1 Introduction 17

3.2 Réalisations et Résultats 17

3.2.1 Screenshots du projet 17

3.2.2 Tests et qualité logicielle 19

3.3 Bilan et Perspectives 20

3.3.1 Bilan du projet 20

3.3.2 Difficultés rencontrées 21

3.3.3 Perspectives et améliorations futures 21

4 Manuel d'Installation et d'Utilisation **23**

4.1 Guide d'Installation 23

4.1.1 Prérequis Système 23

4.1.2 Configuration de la Base de Données 23

4.1.3 Configuration de l'Application 23

4.2 Guide d'Utilisation par Rôle 24

4.2.1 Administrateur 24

4.2.2 Secrétaire 24

4.2.3 Médecin 25

4.2.4 Patient 25

1 Cahier des Charges

1.1 Explicitation des Besoins Clients

Le projet vise à moderniser et digitaliser la gestion d'un cabinet de médecine générale. Le client exprime le besoin d'une plateforme centralisée accessible par quatre types d'utilisateurs distincts.

1.1.1 Besoins Fonctionnels (Spécifications)

- **Administration (RH)** : Gestion exclusive du personnel (médecins et secrétaires) avec contrôle des accès.
- **Gestion Administrative (Secrétariat)** : Centralisation des dossiers patients et gestion du planning global du cabinet.
- **Suivi Médical (Médecin)** : Consultation des antécédents, rédaction de comptes-rendus et génération d'ordonnances en temps réel.
- **Service Patient (Libre-service)** : Autonomie complète pour la prise, le suivi et l'annulation de rendez-vous en ligne.

1.1.2 Besoins Non-Fonctionnels

- **Sécurité** : Hachage des mots de passe (Bcrypt) et authentification sécurisée par jeton (JWT).
- **Disponibilité** : Architecture Client/Serveur avec base de données MySQL persistante.

— **Ergonomie** : Interface responsive (adaptée aux tablettes et smartphones des médecins).

— **Traçabilité** : Journalisation (Logs) des actions critiques.

1.2 Choix du Modèle pour le Cycle de Vie

Le modèle choisi pour ce projet est le **Modèle Incrémental**.

1.2.1 Présentation du modèle choisi

Le modèle incrémental décompose le projet en plusieurs itérations (incréments). Chaque incrément ajoute une fonctionnalité complète au système, de l'analyse à la livraison.

1.2.2 Pourquoi ce modèle est adapté au projet ?

1. **Priorisation des risques** : Nous avons pu livrer le module d'authentification et de sécurité en premier (Incrément 1), garantissant la base du système.
2. **Visibilité** : Le client peut voir évoluer les modules métier (Patients, RDV, Consultations) les uns après les autres.
3. **Flexibilité** : Les ajustements sur la gestion des créneaux ont pu être faits après la mise en place du module patient, sans impacter le module admin.

1.2.3 Description des étapes selon le modèle incrémental

— **Analyse Globale** : Définition des 27 cas d'utilisation et du schéma MySQL.

— **Incrément 1 (Cœur)** : Authentification, rôles et sécurité.

— **Incrément 2 (Admin/Secrétaire)** : CRUD Personnel et Patients.

— **Incrément 3 (Planning)** : Algorithme de réservation et gestion des rendez-vous.

— **Incrément 4 (Médical)** : Consultations, ordonnances et historique.

— **Validation Finale** : Tests unitaires et d'intégration.

1.3 Planning Prévisionnel

Phase	Livrables
Conception & Analyse	Diagrammes UML (Cas d'utilisation, Classes, MPD)
Incrément 1 : Base & Sécurité	API Auth, Middleware JWT, Table Users
Incrément 2 : Gestion Administrative	Dashboard Admin & Secrétaire, CRUD Patients
Incrément 3 : Moteur de RDV	Réservation, Dashboard Patient
Incrément 4 : Suivi Médical	Dashboard Médecin, Prescription, Historique
Tests & Rapport	Rapport de tests, Documentation technique

2 Conception via des AGL et Documentation

Pour la conception, On a utilisé l'AGL ==Entrprise Architect==.

2.0.1 Modélisation UML

La phase de conception a produit un ensemble complet de diagrammes UML couvrant les aspects statiques et dynamiques du système. Ces diagrammes ont servi de référence formelle tout au long du développement.

Diagramme des cas d'utilisation

Identifie l'ensemble des interactions entre les quatre acteurs (patient, médecin, secrétaire, administrateur) et le système.

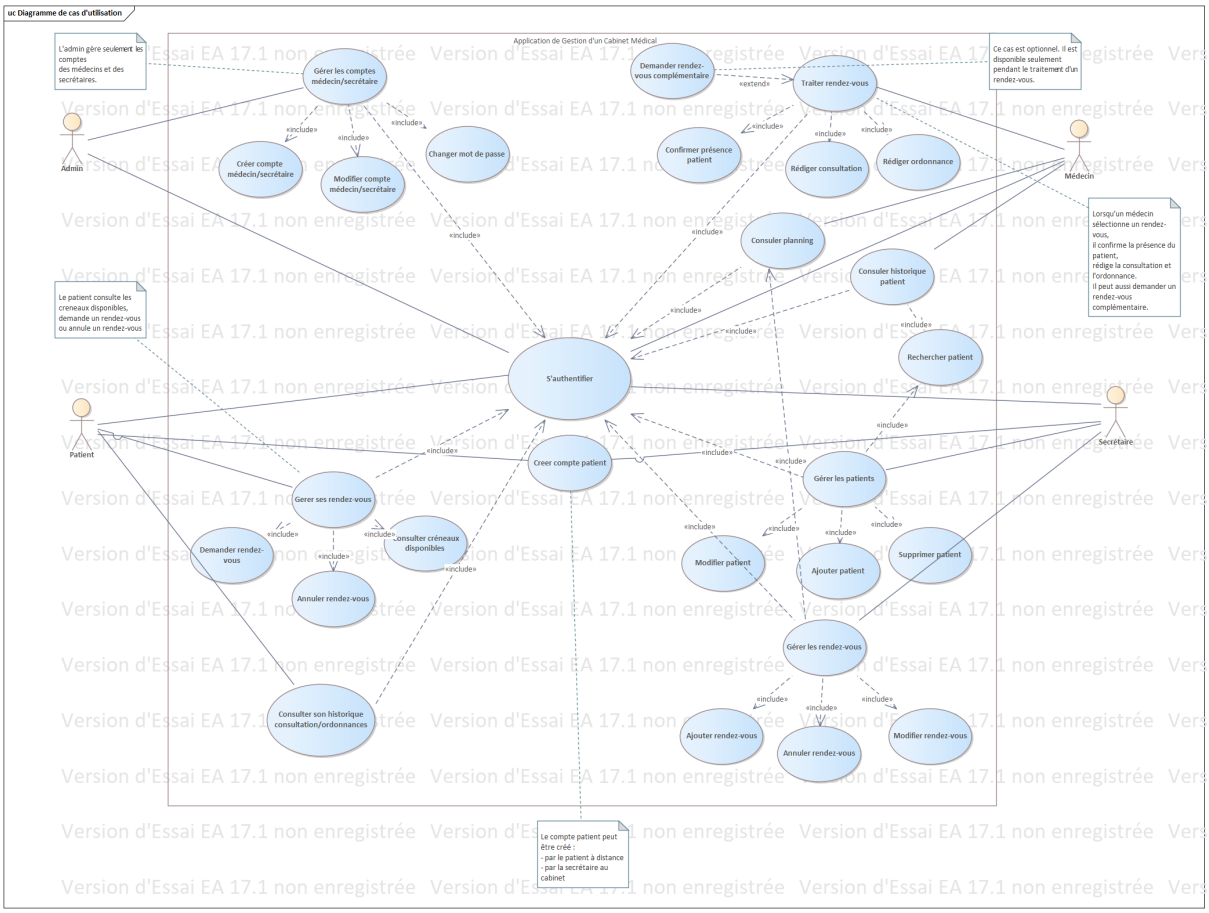


FIGURE 2.1 – Le diagramme de cas d'utilisation

Diagramme de classes

Modélise la structure statique : entités Utilisateur, Patient, Médecin, RendezVous, Consultation, Ordonnance et leurs associations, héritages et multiplicités.

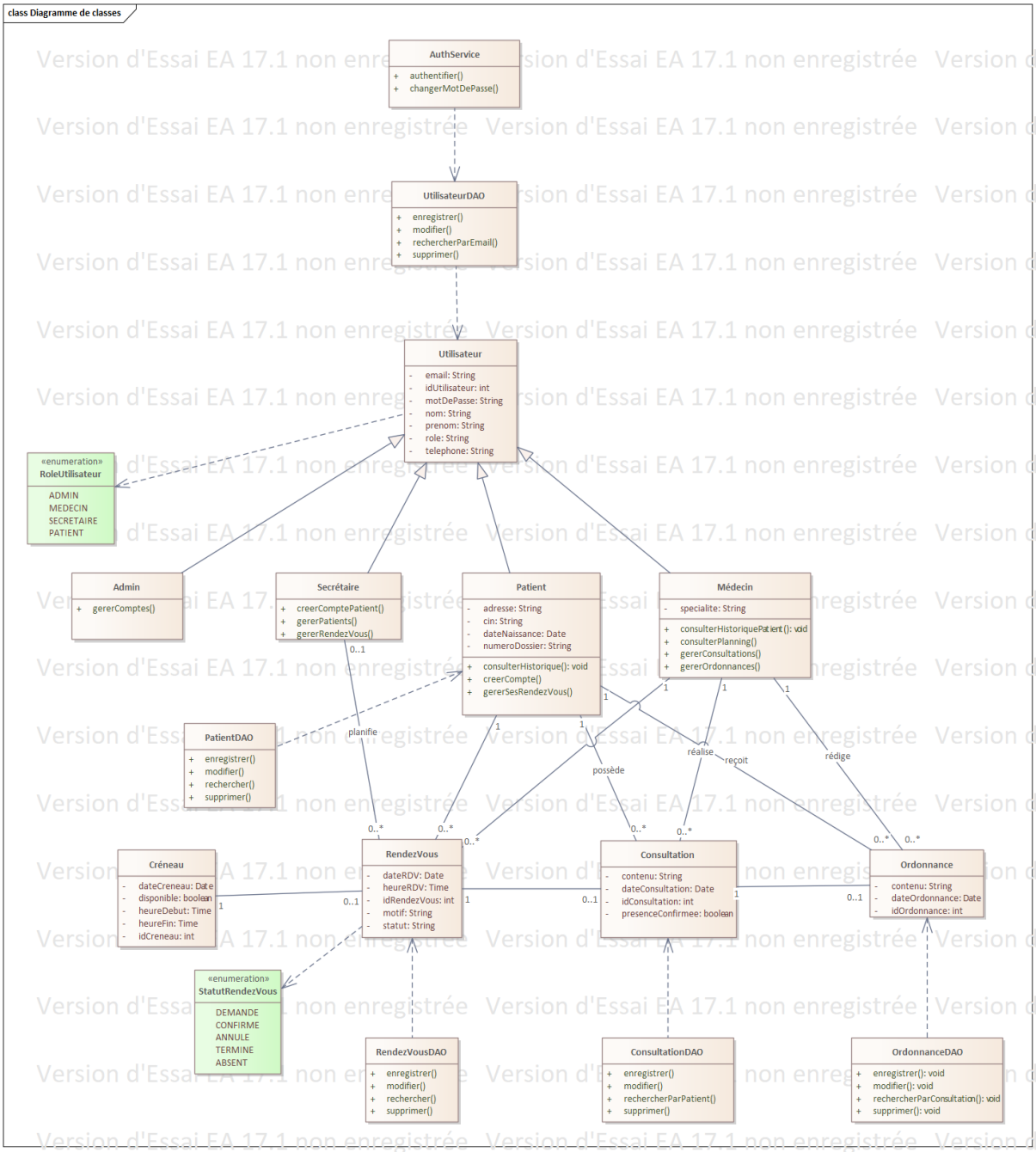


FIGURE 2.2 – ici le titre de l'image|491

Diagramme de composants

Illustre l'architecture logicielle : modules React, serveur Express, couche ORM et base MySQL avec leurs interfaces de communication.

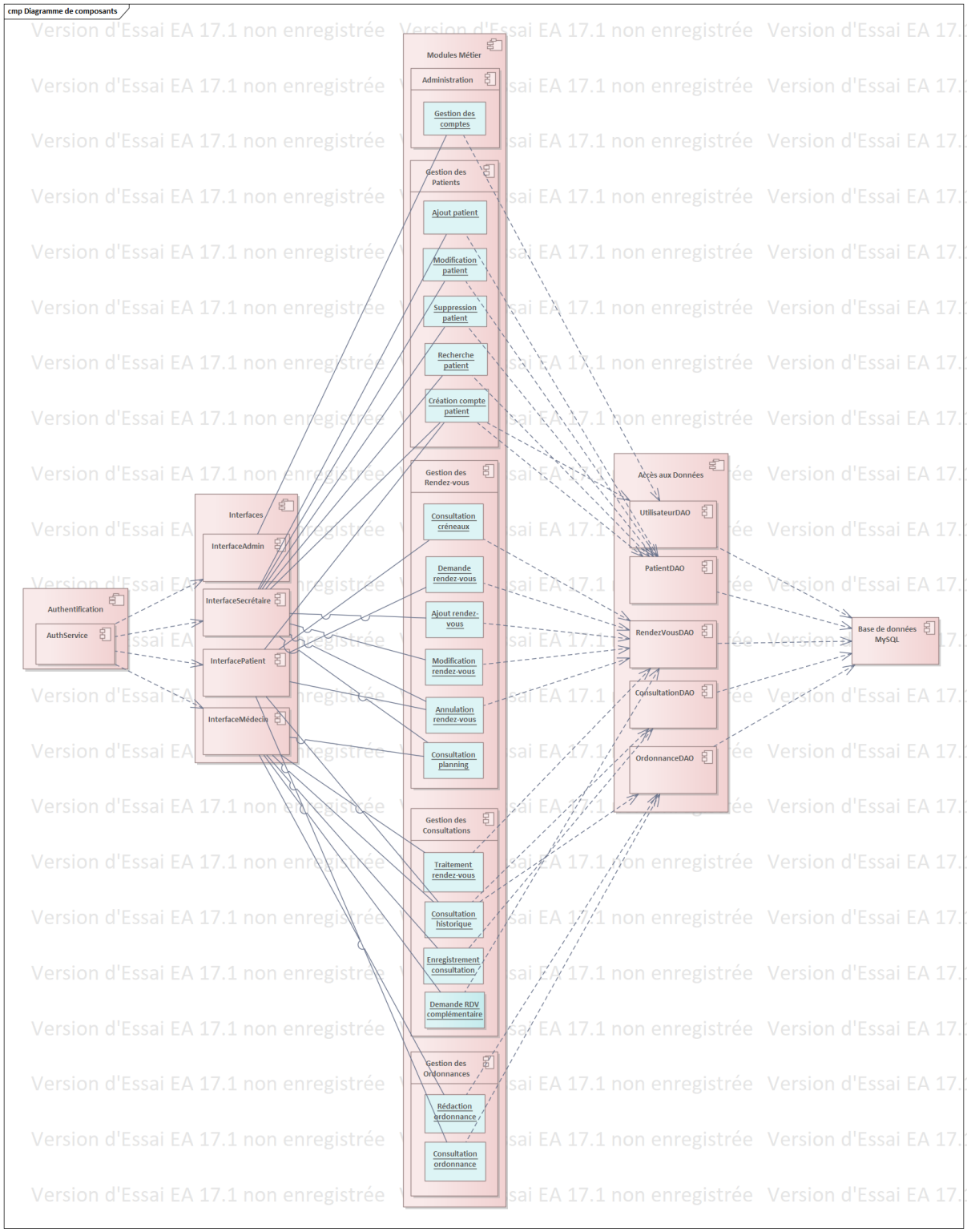
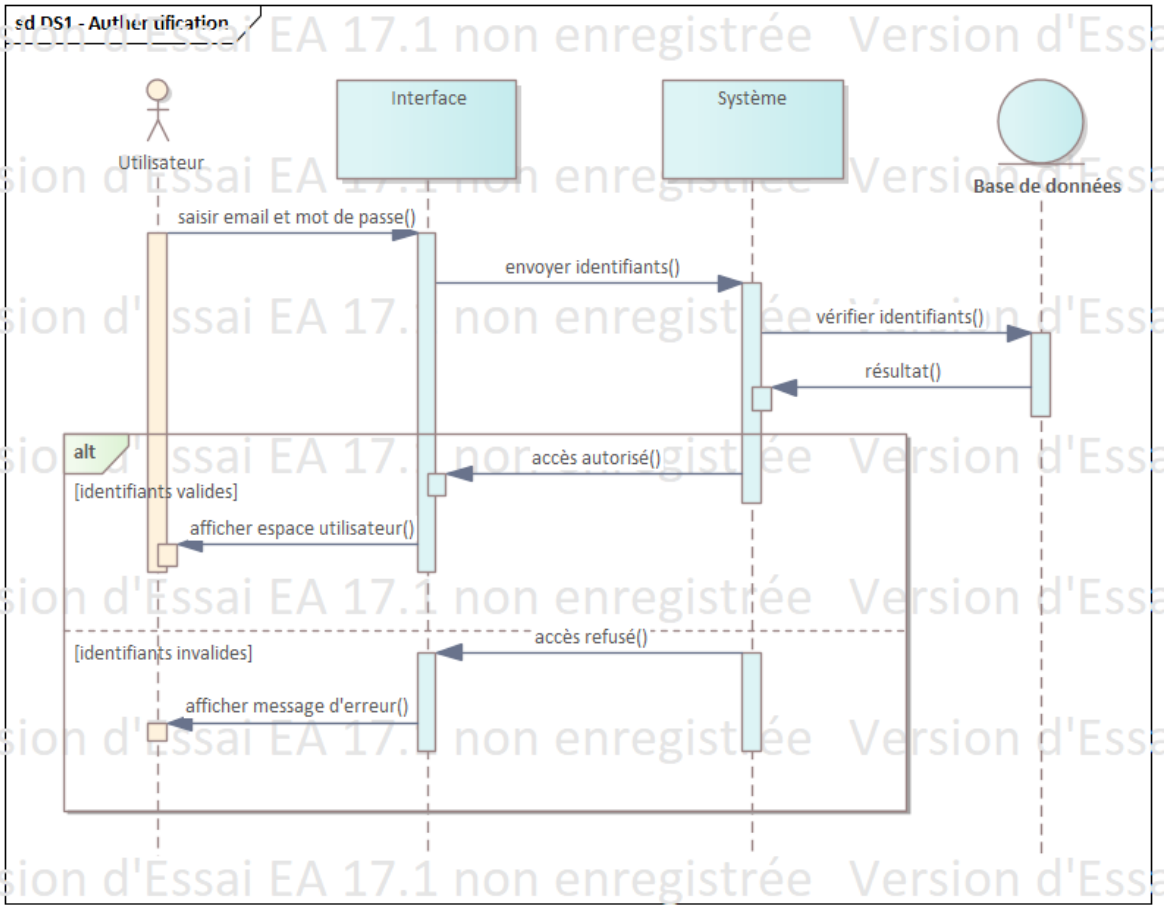


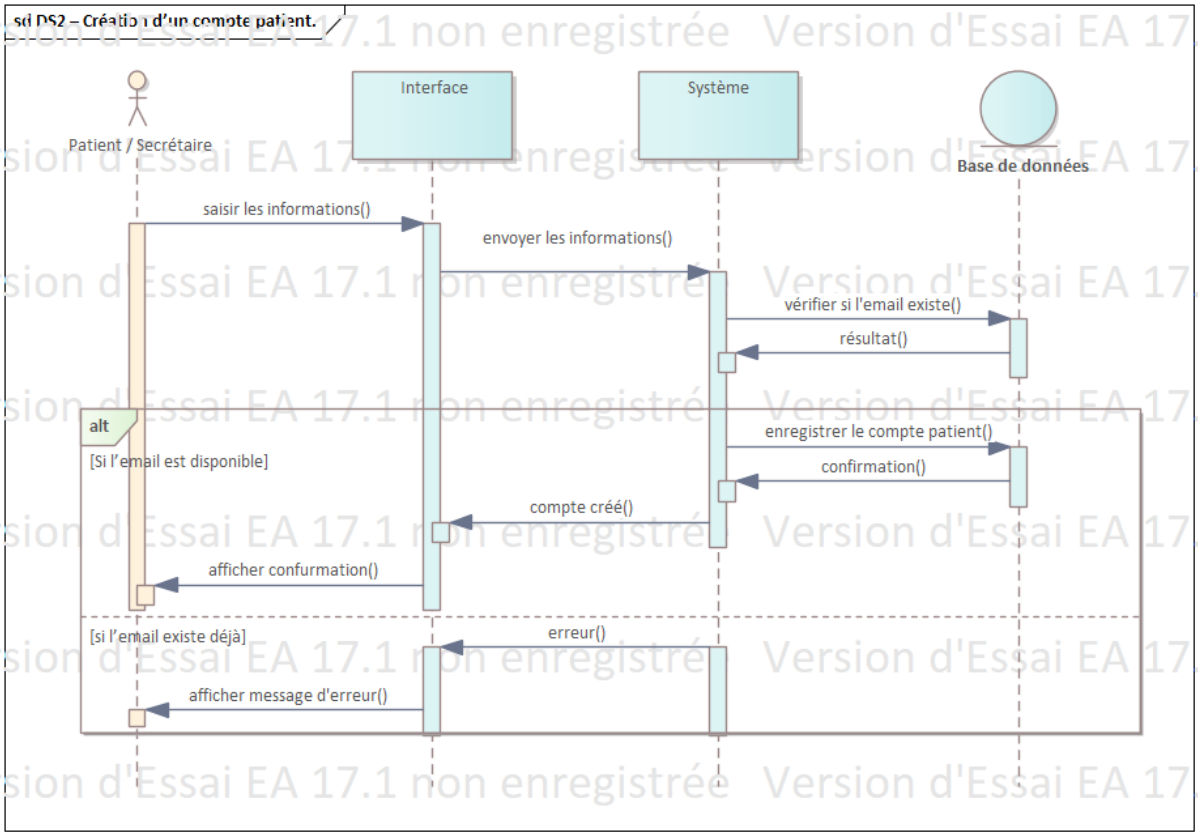
FIGURE 2.3 – Le diagramme de composants|511

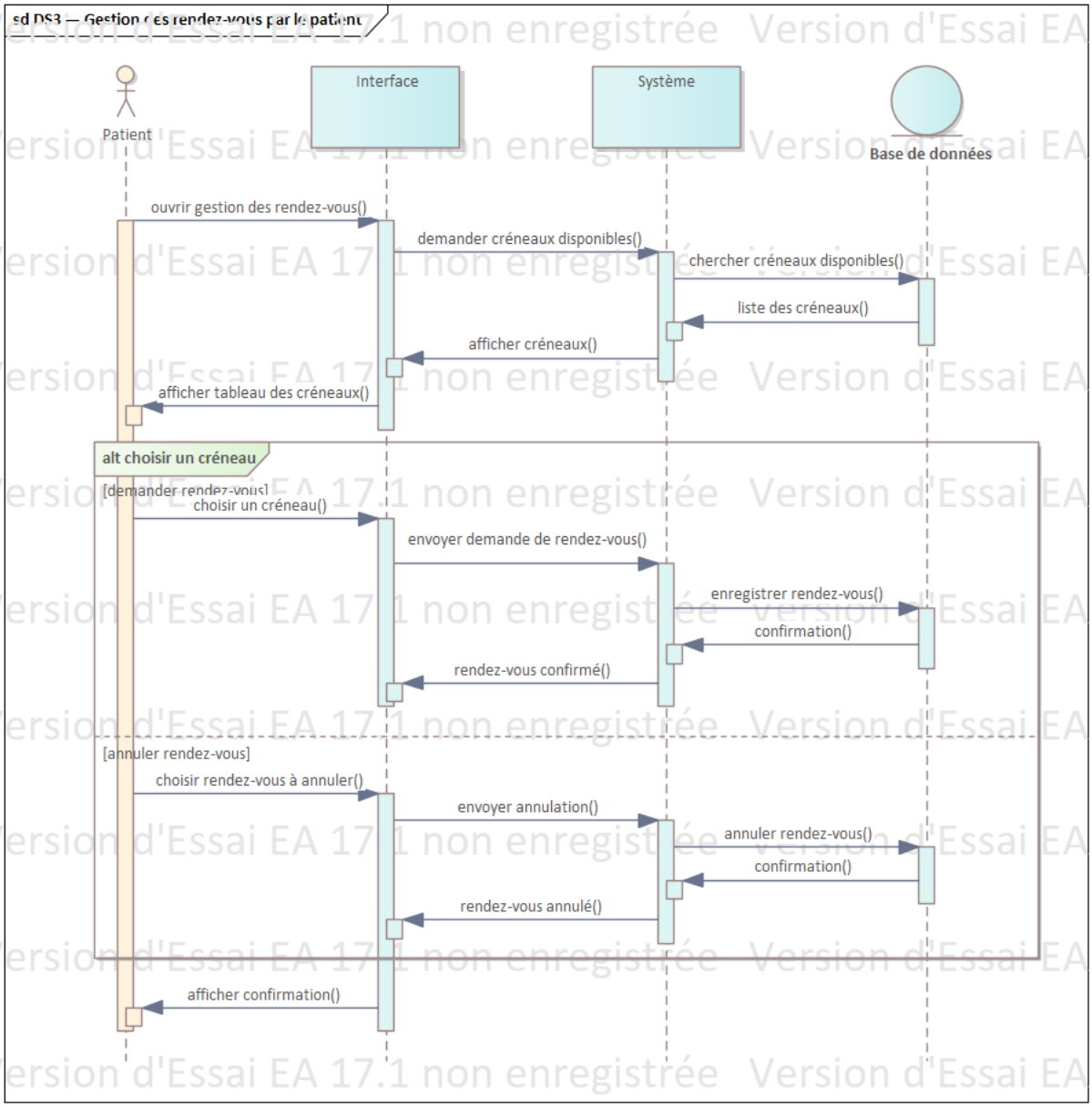
Diagrammes de séquence

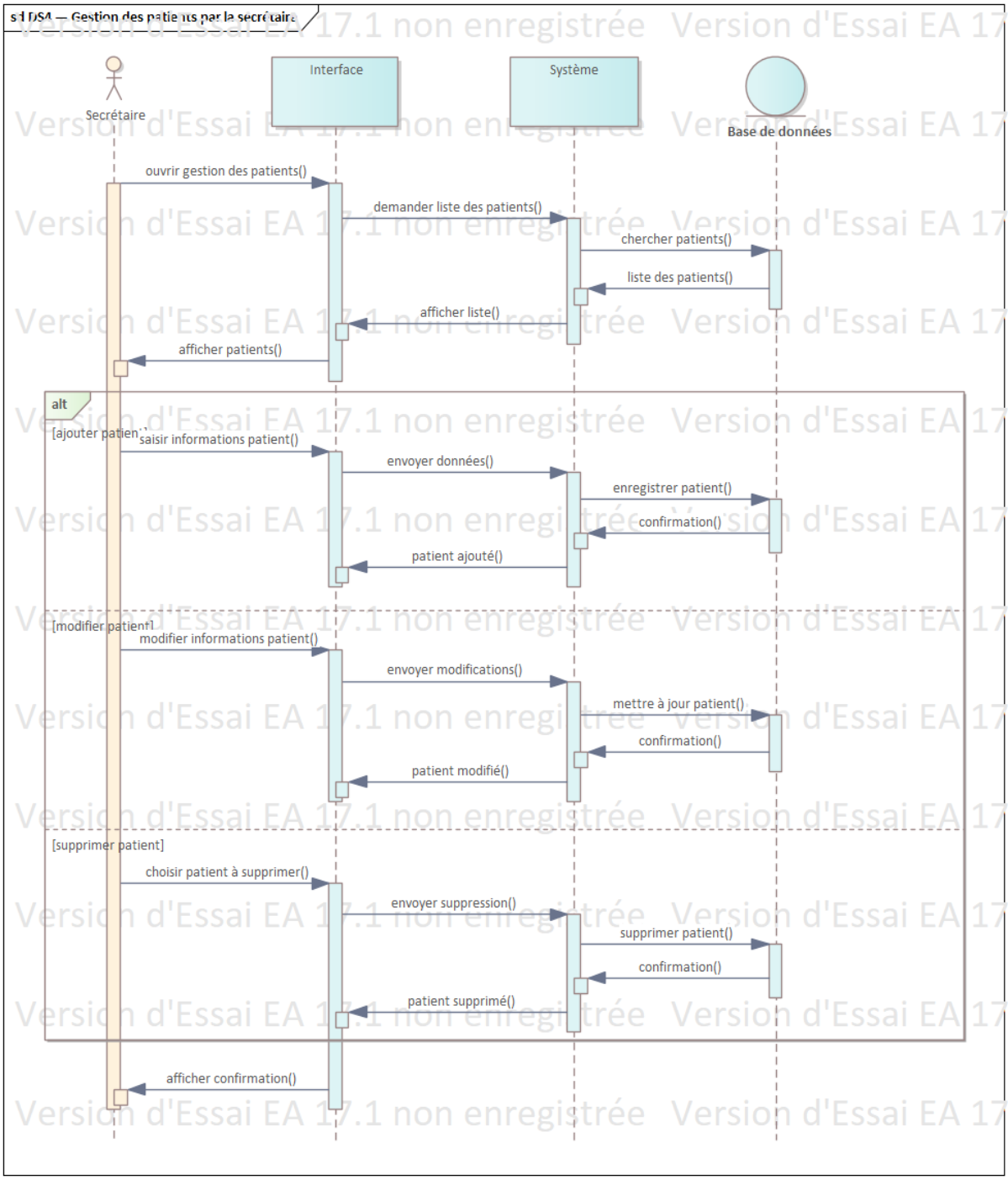
Cinq diagrammes détaillent les interactions dynamiques pour les scénarios fonctionnels critiques du système :

- **DS1** — Authentification (connexion sécurisée avec JWT)
- **DS2** — Création d'un compte patient
- **DS3** — Gestion des rendez-vous par le patient
- **DS4** — Gestion des patients par la secrétaire
- **DS5** — Traitement d'un rendez-vous par le médecin

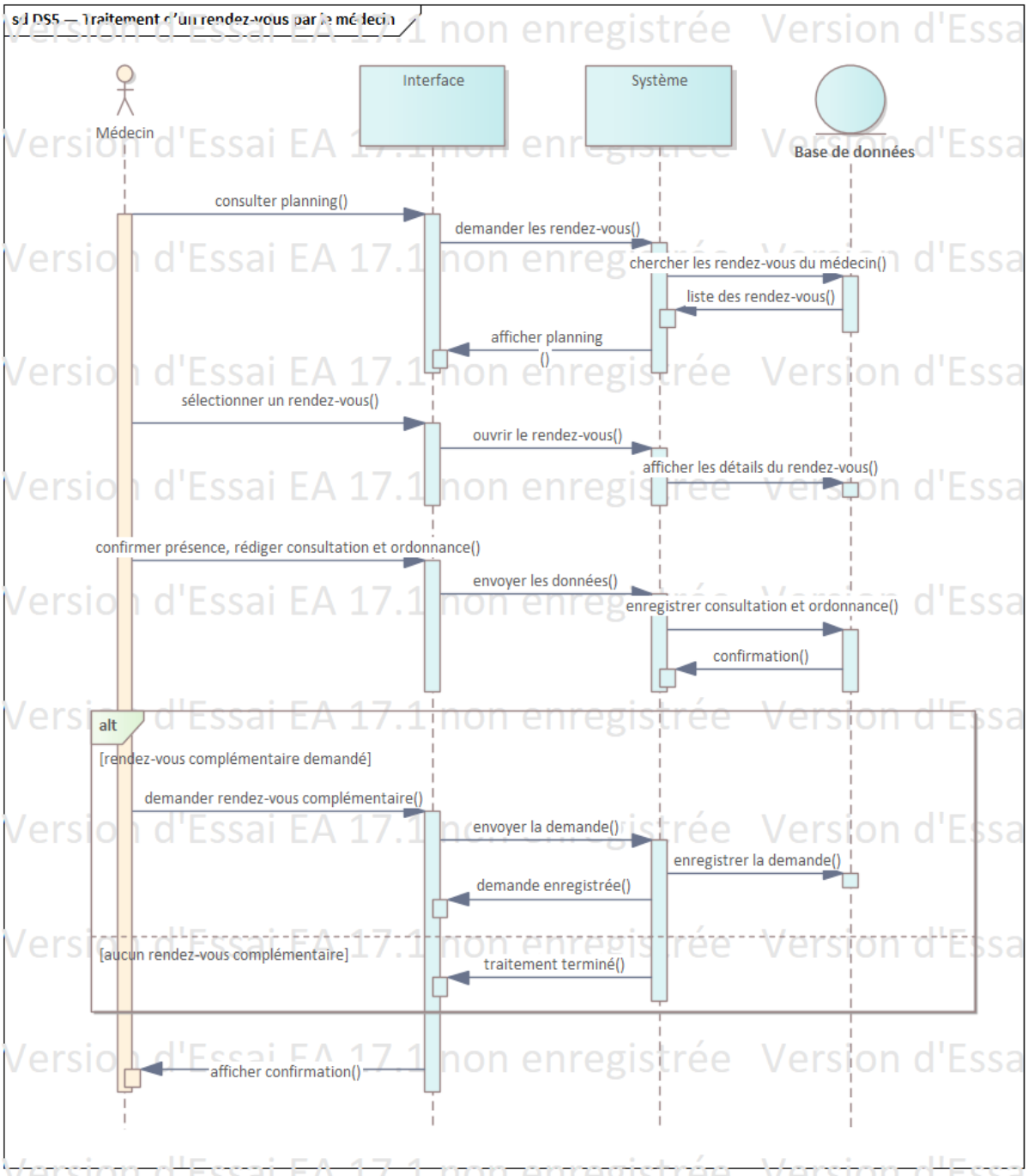



sd DS2 – Création d'un compte patient.
Patient / Secrétaire
Interface
Système
Base de données
saisir les informations()
envoyer les informations()
vérifier si l'email existe()
résultat()
alt
[Si l'email est disponible]
enregistrer le compte patient()
confirmation()
compte créé()
afficher confurmation()
[si l'email existe déjà]
erreur()
afficher message d'erreur()


sd DS3 — Gestion des rendez-vous par le patient
Patient
Interface
Système
Base de données
ouvrir gestion des rendez-vous()
demander créneaux disponibles()
chercher créneaux disponibles()
liste des créneaux()
afficher créneaux()
afficher tableau des créneaux()
alt choisir un créneau
[demander rendez-vous]
choisir un créneau()
envoyer demande de rendez-vous()
enregistrer rendez-vous()
confirmation()
rendez-vous confirmé()
[annuler rendez-vous]
choisir rendez-vous à annuler()
envoyer annulation()
annuler rendez-vous()
confirmation()
rendez-vous annulé()
afficher confirmation()



Version d'Essai EA 17.1 non enregistrée



2.0.2 Modèle physique de données

La base de données MySQL suit une conception relationnelle normalisée. Sept tables constituent le schéma, créées par `database/schema.sql`.

Table	Description	Colonnes clés
users	Tous les utilisateurs (4 rôles)	`id`, `email`, `password_hash`, `role`
patient_details	Informations médicales des patients	`user_id` (FK), `date_of_birth`, `blood_type`, `allergies` (JSON)
doctor_details	Informations professionnelles des médecins	`user_id` (FK), `specialization`, `license_number`
appointments	Rendez-vous planifiés	`patient_id` (FK), `doctor_id` (FK), `date`, `start_time`, `status`
consultations	Comptes-rendus de consultation	`appointment_id` (FK), `is_present`, `report`, `diagnosis`
prescriptions	Ordonnances médicales	`consultation_id` (FK), `medication_name`, `dosage`, `duration`
audit_log	Journal d'audit système	`level` (INFO/WARN/ERROR), `action`, `timestamp`, `ip_address`

Des contraintes d'intégrité référentielle (`ON DELETE CASCADE / RESTRICT`) garantissent la cohérence des données. Des index composites (ex. `idx_apt_slot` sur `doctor_id`, `date`, `start_time`) assurent l'unicité des créneaux et optimisent les requêtes fréquentes.

---

3 Rapport de résultats du projet

3.1 Introduction

Le projet couvre l'intégralité du cycle de développement logiciel : analyse des besoins, modélisation UML, conception de la base de données, implémentation full-stack et tests unitaires.

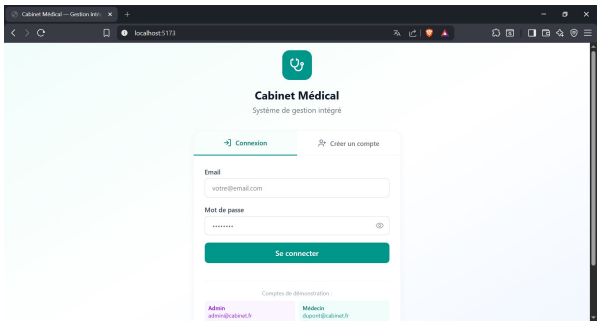
Architecture technique :

L'application adopte une architecture client-serveur avec une séparation stricte entre le frontend et le backend, reliés par une API RESTful.

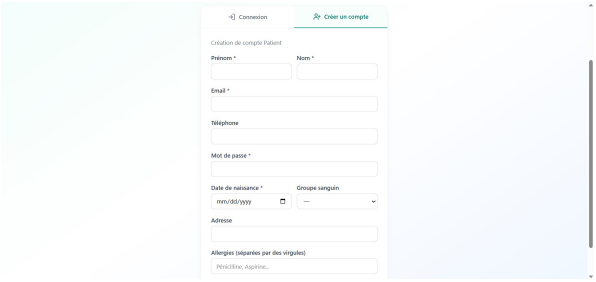
Couche	Technologies	Rôle
Frontend	React · TypeScript · Vite · TailwindCSS	Interface utilisateur réactive
Backend	Node.js · Express.js	API REST, logique métier, JWT
Base de données	MySQL · InnoDB	Persistance relationnelle
Tests	Vitest · Testing Library	Tests unitaires, API, UI

3.2 Réalisations et Résultats

3.2.1 Screenshots du projet

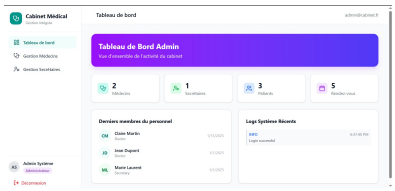


(a) Login page

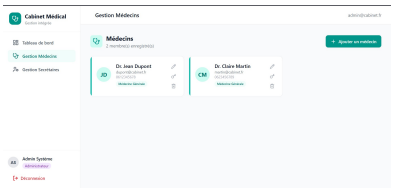


(b) Registre page

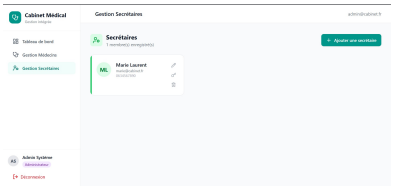
FIGURE 3.1 – Interface Login & Registre



(a) Dashboard

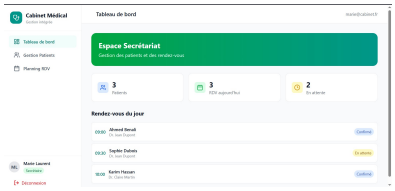


(b) Gestion médecins

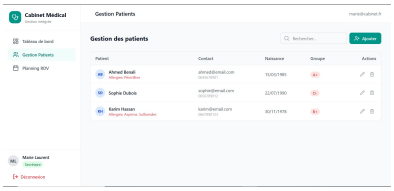


(c) Gestion secrétaires

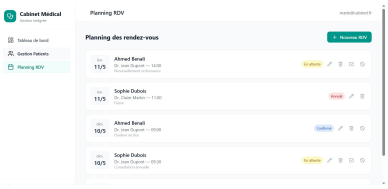
FIGURE 3.2 – Interface Admin



(a) Dashboard



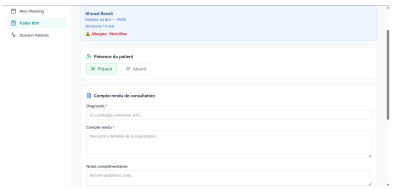
(b) Gestion patients



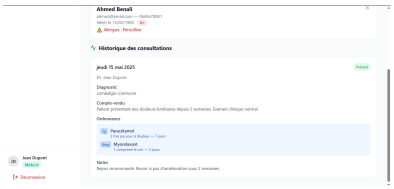
(c) Planning

FIGURE 3.3 – Interface Secrétaire

(a) Dashboard

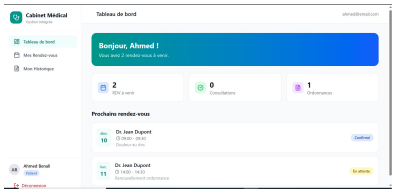


(b) Traiter RDV

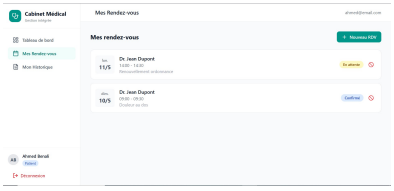


(c) Recherche & Historique

FIGURE 3.4 – Interface Doctor



(a) Dashboard



(b) Mes RDV

(c) Historique médical

FIGURE 3.5 – Interface Patient

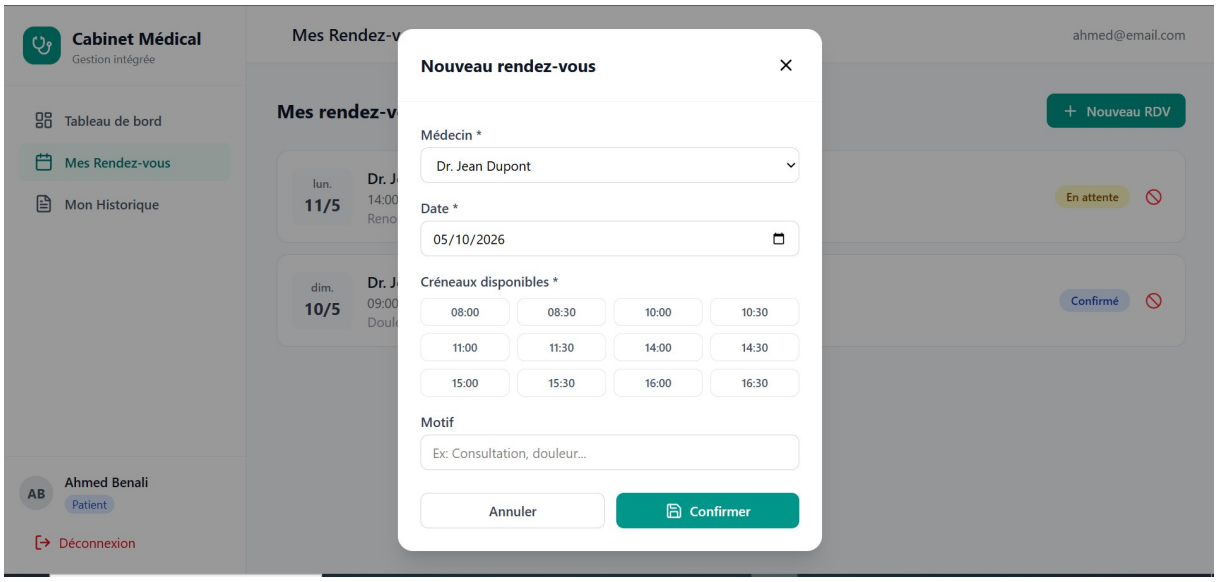


FIGURE 3.6 – Création d'un nouveau rendez-vous

3.2.2 Tests et qualité logicielle

La stratégie de test applique les partitions d'équivalence et les valeurs limites (frontières horaires 11h30/14h00, états terminaux de la machine à états, matrice RBAC complète).

Catégorie	Tests	Portée	Résultat
Backend	29	Logique métier : créneaux, cycle RDV, RBAC, `safeUser`, `changePassword`	100 % (< 2 ms/-test)
API	14	Méthodes HTTP, payloads, codes de réponse	100 %
UI	8	Rendu et interactions du formulaire connexion/inscription	100 %
Logger	5	Fonctionnement et plafonnement mémoire du système de logs	100 %
TOTAL	**56**	Sécurité des données, intégrité fonctionnelle, cohérence UI	**100 %**

```
>npx vitest run --reporter=verbose
```

```
ts 0ms
 ✓ src/test/backend.test.ts > Access Control — role permissions > patient can only manage own
 data 1ms
 ✓ src/test/backend.test.ts > Access Control — role permissions > no role has all permissions
 1ms

 Test Files  4 passed (4)
      Tests  56 passed (56)
   Start at  18:25:54
   Duration  32.11s (transform 2.46s, setup 4.79s, import 6.99s, tests 5.36s, environment 30.
02s)
```

FIGURE 3.7 – Résultat des tests|510

3.3 Bilan et Perspectives

3.3.1 Bilan du projet

Le projet a atteint l'ensemble de ses objectifs initiaux. Une application fonctionnelle, sécurisée et déployée a été livrée dans les délais impartis. Le respect du cycle complet de développement — de l'analyse à la livraison — démontre la maîtrise des principes fondamentaux du Génie Logiciel.

Critère	Objectif	Réalisé
Couverture fonctionnelle	4 rôles, tous modules	100 %
Taux de réussite des tests	90 %	100 % (56/56)
Architecture	Séparation frontend/backend	React + Express + MySQL
Sécurité	JWT + RBAC + hachage	Implémenté et testé
Modélisation UML	UC, Classes, Composants, 5 DS	8 diagrammes livrés

3.3.2 Difficultés rencontrées

Les principaux défis techniques rencontrés et surmontés au cours du projet :

- **Gestion des créneaux horaires :** logique de calcul des disponibilités avec cas limites (créneaux adjacents, chevauchements, frontières horaires 11h30/14h00).
- **Machine à états des rendez-vous :** définition et respect des transitions valides (`pending → confirmed → completed / cancelled / no-show`), blocage des états terminaux.
- **Sécurité de bout en bout :** garantir que le hash du mot de passe n'est jamais exposé côté client, dans les logs ou les réponses API via le helper `safeUser`.
- **Cohérence des types TypeScript :** synchronisation stricte entre les types du frontend et les réponses de l'API REST.

3.3.3 Perspectives et améliorations futures

Axes d'amélioration identifiés pour une version future du système :

- **Notifications par email (rappels RDV)**
- **Export PDF des ordonnances**
- **Tableau de bord avec graphiques**
- **Gestion des congés médecins**

— **Multi-cabinets**

---

4 Manuel d'Installation et d'Utilisation

4.1 Guide d'Installation

4.1.1 Prérequis Système

— **Node.js** : v18.0.0 ou supérieure.

— **MySQL Server** : v8.0.0 ou supérieure.

— **Navigateur Web** : Chrome, Firefox ou Edge (versions récentes).

4.1.2 Configuration de la Base de Données

1. Lancez votre client MySQL (ex : MySQL Workbench ou CMD).
2. Créez une base de données : `CREATE DATABASE cabinet_medical;`.
3. Importez le script `database/schema.sql` fourni dans le dossier du projet.
4. Vérifiez que les tables (users, appointments, etc.) sont bien créées.

4.1.3 Configuration de l'Application

1. Installez les paquets : `npm install`.
2. Créez un fichier `.env` à la racine avec les paramètres suivants :

“‘env

DB_HOST=localhost

DB_USER=root

DB_PASSWORD=votre_mot_de_passe

DB_NAME=cabinet_medical

JWT_SECRET=votre_cle_secrete

PORT=3001

“‘ ### Lancement

— **Mode Développement** : `npm run dev` (Front) et `npx ts-node server/index.ts` (Back).
— **Mode Production** : `npm run build` puis servez le dossier `dist`.

4.2 Guide d'Utilisation par Rôle

4.2.1 Administrateur

— **Objectif** : Gérer les ressources humaines du cabinet.
— **Actions** :

- Créer des comptes pour les nouveaux médecins et secrétaires.

- Modifier les informations du personnel ou réinitialiser leurs mots de passe.

- Consulter les logs système pour surveiller l'activité.

4.2.2 Secrétaire

— **Objectif** : Gestion administrative et flux des patients.
— **Actions** :

- Enregistrer les nouveaux patients qui se présentent au cabinet.

- Gérer le planning global : ajouter, modifier ou annuler des rendez-vous.

- Confirmer les rendez-vous en attente.

4.2.3 Médecin

— **Objectif** : Assurer le suivi médical et les consultations.

— **Actions** :

- Consulter son planning personnel du jour.

- **Traiter un rendez-vous** : Confirmer la présence, saisir le diagnostic, rédiger le compte-rendu et l'ordonnance.

- Rechercher un patient pour consulter l'intégralité de son historique médical.

4.2.4 Patient

— **Objectif** : Accéder aux services de soins de manière autonome.

— **Actions** :

- Créer son compte en ligne.

- Consulter les créneaux disponibles par médecin.

- Demander ou annuler ses propres rendez-vous.

- Consulter son historique personnel (ordonnances et rapports).